

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

Nikhil HR (1WA24CS187)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Nikhil HR (1WA24CS187), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of an OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl No	Experiment Title	Page No

1	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)	1-35
2	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	36-44
3	Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one) a) Rate-Monotonic b) Earliest-Deadline First c) Proportional Scheduling	45-61
4	Write a C program to simulate: (Any one) a) Producer-Consumer problem using semaphores b) Dining-Philosopher's problem	62-75
5	Write a C program to simulate: (Any one) a) Bankers' algorithm for the purpose of deadlock avoidance b) Deadlock Detection	76-94
6	Write a C program to simulate the following contiguous memory allocation techniques. (Any one) a) Worst-Fit b) Best-Fit c) First-Fit	95-105
7	Write a C program to simulate page replacement algorithms. (Any one) a) FIFO b) LRU c) Optimal	106-119

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.

C04	Conduct practical experiments to implement the functionalities of Operating system.
-----	---

Program -1

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS

b) SJF

c) Priority

d) Round Robin (Experiment with different quantum sizes for RR algorithm)

FCFS

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: Nikhil HR\nUSN:1WA24CS187\n\n"); int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &at[i]);

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &bt[i]);

    int t = 0;
    for(i = 0; i < n; i++)
    {
```

```

    if(t < at[i])
        t = at[i];

    ct[i] = t + bt[i];
    t = ct[i];

    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i]; }

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++)
{ printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;

```

```

Name: NIKHIL HR
USN:1WA24CS187

Enter number of processes: 4
Enter Arrival Time:
0 0 0 0
Enter Burst Time:
7 3 4 6

Process AT      BT      CT      TAT      WT
P1      0      7      7      7      0
P2      0      3     10     10     7
P3      0      4     14     14     10
P4      0      6     20     20     14
PS C:\Users\Usere\Documents\LAB_REPORT>

```

#FCFS

#include <stdio.h>

int main () {

int n, i;

printf("Enter total of process: ");

scanf("%d", &n);

int at[n], bt[n], ct[n], tat[n], wt[n];

float avg_tat = 0; avg_wt = 0;

for (i=0; i<n; i++) {

printf("Process (%d)", i+1);

printf("Arrival Time ");

scanf("%d", &at[i]);

printf("Burst Time ");

scanf("%d", &bt[i]);

ct[0] = at[0] + bt[0];

for (i=0; i<n; i++) {

if (ct[i-1] < at[i])

ct[i] = at[i] + bt[i];

else

ct[i] = ct[i-1] + bt[i];

}

for (i=0; i<n; i++) {

tat[i] = ct[i] - at[i];

wt[i] = tat[i] - bt[i];

avg_tat += tat[i];

avg_wt += wt[i];

}

avg_tat = avg_tat / n;

avg_wt = avg_wt / n;

```

printf("Process  AT  BT  CT  TAT  WT);

for (i=0; i<n; i++)
    printf(" %d  %d  %d  %d  %d  %d,
           it, at[i], bt[i], ct[i], tat[i], wt[i]);

}

printf(" Avg TAT: %.2f ", avg-tat);
printf(" Avg WT : %.2f ", avg-wt);

return 0;
}

```

O/P:

Enter total no. of process : 4

Process [1]

AT : 0

BT : 2

Process : [2]

AT : 1

BT : 2

Process [3]

AT : 5

BT : 3

Processes [4]

AT : 6

BT : 4

SJF (Non – Preemptive)

CODE:

```
#include <stdio.h>

int main()
{
printf("Name:Nikhil HR\nUSN:1WA24CS187\n\n");
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n];
    int done[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
    { scanf("%d", &at[i]);
      done[i] = 0; }

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &bt[i]);

    int t = 0, completed = 0;

    while(completed < n)
    { int min = 9999;
      int index = -1;

      for(i = 0; i < n; i++)
```

```

{ if(at[i] <= t && done[i] == 0)
    { if(bt[i] < min)
        { min = bt[i];
          index = i;
        }
    }
}

if(index == -1)
{ t++;
}
else
{ ct[index] = t + bt[index];
  t = ct[index];

  tat[index] = ct[index] - at[index];
  wt[index] = tat[index] - bt[index];

  done[index] = 1;
  completed++;
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n"); for(i = 0; i < n; i++)
{ printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

Result:

Name: Nikhil HR

USN:1WA24CS187

Enter number of processes: 4

Enter Arrival Time:

0 0 0 0

Enter Burst Time:

7 3 4 6

Process	AT	BT	CT	TAT	WT
P1	0	7	20	20	13
P2	0	3	3	3	0
P3	0	4	7	7	3
P4	0	6	13	13	7

PS C:\Users\Usere\Documents\LAB_REPORT> █

SJF- Non Preemptive

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i, j, curr = 0, comp = 0;
```

```
    printf("Enter no. of process : ");
```

```
    scanf("%d", &n);
```

```
    int pid[n], at[n], bt[n], ct[n], tot[n], ut[n],  
        fin[n];
```

```
    for (i = 0; i < n; i++)
```

```
        finished[i] = 0;
```

```
    for (i = 0; i < n; i++)
```

```
        printf("Enter AT & BT for pid %d:", i);
```

```
        pid[i] = i + 1;
```

```
        scanf("%d %d", &at[i], &bt[i]);
```

```
}
```

```
while (!completed) {
```

```
    int idx = -1;
```

```
    int min_bt = 100000;
```

```
    for (i = 0; i < n; i++)
```

```
        if (at[i] < curr && fin[i] == 0)
```

```
            if (bt[i] < min_bt)
```

```
                min_bt = bt[i];
```

```
                idx = i;
```

```
    }
```

```
    }
```

```
    if (idx == -1)
```

```
        curr++;
```



```

else{
    ct[idx] = curr + bt[idx];
    tat[idx] = ct[idx] - at[idx];
    wt[idx] = tat[idx] - bt[idx];

    curr = ct[idx];
    fin[idx] = 1;
    comp++;
}

printf("PID  AT  BT  CT  TAT  WT");
for(i=0; i<n; i++){
    printf("%d  %d  %d  %d  %d  %d",
           pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
}
return 0;
}

```

O/P:
 Enter no. of process : 4
 Enter AT and BT for P1: 0
 6 and BT for P2: 0
 Enter AT and BT for P3: 0
 8 and BT for P4: 0
 Enter AT and BT for P4: 0
 7 and BT for P4: 0
 3

PID	AT	BT	CT	TAT	WT
P1	0	6	9	9	3
P2	0	8	24	24	16
P3	0	7	16	16	9
P4	0	3	3	3	0

SRTF

CODE:

```
#include <stdio.h>
```

```

int main()
{
printf("Name:Nikhil HR\nUSN:1WA24CS187\n\n");

int n, i;


printf("Enter number of processes: ");
scanf("%d", &n);


int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];


printf("Enter Arrival Time:\n");
for(i = 0; i < n; i++)
    scanf("%d", &at[i]);


printf("Enter Burst Time:\n");
for(i = 0; i < n; i++)
{ scanf("%d", &bt[i]);
rt[i] = bt[i]; }


int t = 0, completed = 0;


while(completed < n)
{ int min = 9999;
int index = -1;


for(i = 0; i < n; i++)
{ if(at[i] <= t && rt[i] > 0)
{ if(rt[i] < min)
{ min = rt[i];
index = i;
}
}
}
}

```

```

    }
}

if(index == -1)
{ t++;
}
else
{
    rt[index]--;
    t++;

    if(rt[index] == 0)
    { ct[index] = t;
      completed++;
      ;

      tat[index] = ct[index] - at[index];
      wt[index] = tat[index] - bt[index];
    }
}
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{ printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

Result:

Result:

Name: Nikhil HR

USN:1WA24CS187

Enter number of processes: 4

Enter Arrival Time:

0 0 0 0

Enter Burst Time:

7 3 4 6

Process	AT	BT	CT	TAT	WT
P1	0	7	20	20	13
P2	0	3	3	3	0
P3	0	4	7	7	3
P4	0	6	13	13	7

PS C:\Users\Usere\Documents\LAB_REPORT> █

SJF preemptive

```
#include <stdio.h>
```

```
int main()
```

```
int n, i, time = 0, comp = 0, short = -1;
```

```
printf("%d", &n);
```

```
int pid[n], at[n], bt[n], rt[n], ct[n], tat[n], wt[n];
```

```
int min_rt;
```

```
for (i = 0; i < n; i++)
```

```
pid[i] = i + 1;
```

```
printf("Enter AT and BT for %d",
```

```
scanf("%d %d", &at[i], &bt[i]);
```

```
rt[i] = bt[i];
```

```
}
```

```
while (comp < n)
```

```
short = -1;
```

```
min_rt = 9999;
```

```
for (i = 0; i < n; i++)
```

```
if (at[i] <= time && rt[i] > 0 && rt[i] < min_rt)
```

```
min_rt = rt[i];
```

```
short = i;
```

```
}
```

```
}
```

```
if (shortest == -1)
```

```
time ++;
```

```
continue;
```

```
}
```

```
rt[shortest] --;
```

```
time ++;
```



```

if (rt[short] == 0) {
    comp++;
    ct[short] = time;
    tat[short] = ct[short] - at[short];
    wt[short] = tat[short] - bt[short];
}

float avg-wt = 0, avg-tat = 0;

printf("PID  AT  BT  CT  TAT  WT\n");

for (i = 0; i < n; i++) {
    printf("%d  %d  %d  %d  %d  %d\n",
           pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);

    avg-wt += wt[i];
    avg-tat += tat[i];
}

avg-wt /= n;
avg-tat /= n;

printf("Avg WT = %.2f\n", avg-wt);
printf("Avg TAT = %.2f\n", avg-tat);

return 0;

```

no. of process : 5

AT & BT for P1: 2

AT and BT for P2: 1

AT and BT for P3: 4

AT and BT for P4: 0

AT and BT for P5: 2

AT	BT	CT	TAT	WT
2	1	3	1	0
1	5	11	10	5
4	1	5	1	0
0	6	16	16	10
2	3	7	5	2

WT : 3.40

TAT : 6.60

PRIORITY NON-PREEMPTIVE

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: Nikhil HR\nUSN:1WA24CS187\n\n");
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], ct[n], tat[n], wt[n];
    int done[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
    { scanf("%d", &at[i]);
      done[i] = 0; }

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &bt[i]);

    printf("Enter Priority:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pr[i]);

    int t = 0, completed = 0;
    while(completed < n)
    { int min = 9999;
      int index = -1;
```



```

for(i = 0; i < n; i++)
{ if(at[i] <= t && done[i] == 0)
    { if(pr[i] < min)
        { min = pr[i];
          index = i;
        }
    }
}

if(index == -1)
{ t++;
}
else
{ ct[index] = t + bt[index];
  t = ct[index];

  tat[index] = ct[index] - at[index];
  wt[index] = tat[index] - bt[index];

  done[index] = 1;
  completed++;
}
}

printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{ printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}

return 0;

```

}

Result:

```
Result:

Name: Nikhil HR
USN:1WA24CS187

Enter number of processes: 5
Enter Arrival Time:
0 2 3 4 5 6
Enter Burst Time:
3 2 5 4 1
Enter Priority:
5 3 2 4 1

Process AT      BT      PR      CT      TAT      WT
P1      0       6       1       6       6       0
P2      2       3       5      20      18      15
P3      3       2       3      13      10       8
P4      4       5       2      11       7       2
P5      5       4       4      17      12       8
PS C:\Users\Users\Documents\LAB_REPORT>
```

Non-preemptive Priority Scheduling

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int n;
```

```
    int pid[20], at[20], bt[20], pr[20];
```

```
    int ct[20], tat[20], wtt[20];
```

```
    int comp[20] = {0};
```

```
    int curr = 0, comp = 0;
```

```
    int i;
```

```
    printf("Enter no. of process : ");
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("Process ID : ");
```

```
        printf("Enter Process ID : ");
```

```
        scanf("%d", &pid[i]);
```

```
        printf("Enter AT : ");
```

```
        scanf("%d", &at[i]);
```

```
        printf("Enter BT : ");
```

```
        scanf("%d", &bt[i]);
```

```
        printf("Enter Priority : ");
```

```
        scanf("%d", &pr[i]);
```

```
    }
```

```
    while (comp < n) {
```

```
        int high = 9999;
```

```
        int selec = -1;
```

```
        for (i = 0; i < n; i++) {
```

```
            if (at[i] <= curr & comp[i] == 0) {
```

```
                if (pr[i] < high) {
```

```

highest_pr = pr(i) ;
slec = i ;

y
y
y
if (slec == -1) <
    curr ++;
y
else <
    ct[slec] = curr + bt[slec];
    tat[slec] = ct[slec] - at[slec];
    wt[slec] = tat[slec] - bt[slec];

    curr = ct[slec];
    comp[slec] = 1;
    comp ++;
y
y
printf("Process   AT   BT   Pr   CT   Tat\n");
for (i=0; i<n; i++) <
    printf("%d   %d   %d   %d   %d\n",
        pid[i], at[i], bt[i], pr[i], ct[i],
        wt[i]);
y
return 0;
y

```

O/p:

Enter no. of process : 5

P1:

Enter P ID :

Enter AT : 0

Enter BT : 3

Enter Priority : 5

Q10:

Enter PID: 2

Enter AT: 2

Enter BT: 2

Enter PI: 3

Enter PID: 3

Enter AT: 3

Enter BT: 5

Enter Priority: 2

Enter PID: 5

Enter AT: 6

Enter BT: 1

Enter Priority: 1

Process	AT	BT	Priority	CT	TAT
P1	0	3	5	3	3
P2	2	2	4	11	9
P3	3	5	3	8	5
P4	4	4	2	15	11
P5	6	1	1	9	3

PRIORITY PREEMPTIVE

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: Nikhil HR\nUSN:1WA24CS187\n\n");
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], rt[n];
    int ct[n], tat[n], wt[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &at[i]);

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
    { scanf("%d", &bt[i]);
      rt[i] = bt[i]; }

    printf("Enter Priority:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pr[i]);

    int t = 0, completed = 0;
    while(completed < n)
```

```

{ int min = 9999;
  int index = -1;

  for(i = 0; i < n; i++)
  { if(at[i] <= t && rt[i] > 0)
    { if(pr[i] < min)
      { min = pr[i];
        index = i;
      }
    }
  }

  if(index == -1)
  { t++;
  }
  else
  {
    rt[index]--;
    t++;

    if(rt[index] == 0)
    { ct[index] = t; tat[index] =
      ct[index] - at[index]; wt[index] =
      tat[index] - bt[index];
      completed++;
    }
  }
}

printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)

```



```

    { printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
    }

    return 0;
}

```

Result:

```

Result:

Name: Nikhil HR
USN:1WA24CS187

Enter number of processes: 7
Enter Arrival Time:
6 4 1 3 0 5 10
Enter Burst Time:
5 1 2 4 8 6 1
Enter Priority:
6 5 4 4 3 2 1

Process AT      BT      PR      CT      TAT      WT
P1      6      5      6      27      21      16
P2      4      1      5      22      18      17
P3      1      2      4      17      16      14
P4      3      4      4      21      18      14
P5      0      8      3      15      15      7
P6      5      6      2      12      7      1
P7     10      1      1      11      1      0
PS C:\Users\User\Documents\LAB_REPORT>

```

Priority Scheduling Pre-emptive

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, choice;
```

```
    int pid[20], pr[20], at[20], bt[20], rt[20];
```

```
    int ct[20], tat[20], wt[20];
```

```
    int comp = 0, curr = 0;
```

```
    int i, sel, pri;
```

```
    float avg-wt = 0, avg-tat = 0;
```

```
    printf("Enter no. of process : ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter priority : ");
```

```
    scanf("%d", &choice);
```

```
    printf("Enter priority : ");
```

```
    for (i = 0; i < n; i++)
```

```
        scanf("%d", &pr[i]);
```

```
    printf("Enter BT : ");
```

```
    for (i = 0; i < n; i++)
```

```
        scanf("%d", &bt[i]);
```

```
    printf("Enter BT : ");
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &bt[i]);
```

```
        rt[i] = bt[i];
```

```
        pid[i] = i+1;
```

```
    }
```

```
    while (comp < n)
```

```
        selected = -1;
```

```

if(choice == 2)
    pr = 9999;
else
    pr = -1;

for(i=0; i<n; i++)
    if(at[i] <= curr && rt[i] > 0)
        if(choice == 2)
            if(pr[i] < best - pr)
                pr = pr[i];
                sel = i;
            else
                if(pr[i] > pr)
                    b - pr = pr[i];
                    sel = i;
        }
    }

if(sel == -1)
    curr ++;
else
    rt[sel] --;
    curr ++;
    if(rt[sel] == 0)
        ct[sel] = curr;
        tat[sel] = ct[sel] - at[sel];
        wt[sel] = tat[sel] - bt[sel];
        avg - tat += tat[sel];
        avg - wt += wt[sel];
        comp ++;

```

Order	BT:						
8							
2							
4							
1							
6							
5							
1							
Priority	scheduling (pre-emptive):						
PID	Prior	BT	BT	CT	TAT	WT	RT
P1	3	0	8	15	15	7	7
P2	4	1	2	17	16	14	14
P3	4	3	4	21	18	14	14
P4	5	4	1	22	18	17	17
P5	2	5	6	12	7	1	1
P6	6	6	5	27	21	16	16
P7	1	10	1	11	1	0	0
Avg TAT : 13.714							
Avg WT : 9.857							

ROUND ROBIN

CODE:

```
#include <stdio.h>
```

```

int main()
{
printf("Name: Nikhil HR\nUSN:1WA24CS187\n\n");
    int n, tq, i;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Time Quantum: ");
    scanf("%d",&tq);

    int pid[n], at[n], bt[n], rt[n];
    int wt[n], tat[n], ct[n], resp[n];
    int queue[100]; int front=0,
    rear=0;

    for(i=0;i<n;i++)
    { printf("Enter Process ID, Arrival Time, Burst Time:\n");
        scanf("%d %d %d",&pid[i],&at[i],&bt[i]);

        rt[i] = bt[i];
        wt[i] = 0;
        tat[i] = 0;
        resp[i] = -1;
    }
    int current_time = 0;
    int completed = 0;
    int visited[n];

    for(i=0;i<n;i++)
        visited[i] = 0;

```

```

while(completed < n)
{ for(i=0;i<n;i++)
    { if(at[i] <= current_time && visited[i] == 0)
        { queue[rear++] = i;
          visited[i] = 1;
        }
    }
}

if(front == rear)
{ current_time++;
  continue; }

int p = queue[front++];

if(resp[p] == -1) resp[p] =
    current_time - at[p];

if(rt[p] > tq)
{ current_time += tq;
  rt[p] -= tq;

  for(i=0;i<n;i++)
  { if(at[i] <= current_time && visited[i] == 0)
      { queue[rear++] = i;
        visited[i] = 1;
      }
  }

  queue[rear++] = p;
}
else

```

```

    { current_time += rt[p];
      rt[p] = 0;

      ct[p] = current_time;
      tat[p] = ct[p] - at[p];
      wt[p] = tat[p] - bt[p];
      completed++;
    }
  }

  float avg_wt=0, avg_tat=0;

  printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

  for(i=0;i<n;i++)
  { printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
    pid[i],at[i],bt[i],ct[i],tat[i],wt[i],resp[i]);

    avg_wt += wt[i];
    avg_tat += tat[i]; }

  printf("\nAverage Waiting Time = %.2f",avg_wt/n);
  printf("\nAverage Turnaround Time = %.2f",avg_tat/n);

  return 0;
}

```

Result:

```
Enter Process ID, Arrival Time, Burst Time:
1 0 5
Enter Process ID, Arrival Time, Burst Time:
2 1 3
Enter Process ID, Arrival Time, Burst Time:
3 2 1
Enter Process ID, Arrival Time, Burst Time:
4 3 2
Enter Process ID, Arrival Time, Burst Time:
5 4 3
```

PID	AT	BT	CT	TAT	WT	RT
1	0	5	13	13	8	0
2	1	3	12	11	8	1
3	2	1	5	3	2	2
4	3	2	9	6	4	4
5	4	3	14	10	7	5

```
Average Waiting Time = 5.80
Average Turnaround Time = 8.60
```


Round Robin

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, tq;
```

```
    int at(20), bt(20), rt(20);
```

```
    int ct(30), tat(20), wt(20);
```

```
    int i, time = 0, remain, flag = 0;
```

```
    float avg-wt = 0, avg-tat = 0;
```

```
    printf("Enter no. of process : ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter TA : ");
```

```
    scanf("%d", &tq);
```

```
    printf("Enter AT : ");
```

```
    for (i=0; i<n; i++)
```

```
        scanf("%d", &at[i]);
```

```
    printf("Enter burst time : ");
```

```
    for (i=0; i<n; i++) {
```

```
        scanf("%d", &bt[i]);
```

```
        rt[i] = bt[i];
```

```
    }
```

```
    remain = n;
```

```
    for (time = 0, i=0, remain != 0; i < n; i++) {
```

```
        if (rt[i] == tq && rt[i] > 0) {
```

```
            time += rt[i];
```

```
            rt[i] = 0;
```

```
            flag = 1;
```

```
        }
```

```
else if (rt[i] > 0) {
```

```
    rt[i] -= tq;
```

```
    time += tq;
```

```
}
```

```
if (rt[i] == 0 && flag == 1) {
```

```
    remain --;
```

```
    ct[i] = time;
```

```
    tat[i] = time - at[i];
```

```
    wt[i] = tat[i] - bt[i];
```

```
    avg-wt += wt[i];
```

```
    avg += tat[i];
```

```
    flag = 0;
```

```
}
```

```
if (i == n-1)
```

```
    i = 0
```

```
else if (at[i+1] <= time)
```

```
    i++
```

```
else
```

```
    i = 0;
```

```
}
```

```
printf("RPS scheduln");
```

```
printf("PID  BT  BT  CT  TAT  WT  RT");
```

```
for (i=0; i<n; i++) {
```

```
    printf("%d %d %d %d %d %d %d",
```

```
        i+1, at[i], bt[i], ct[i], tat[i], wt[i],
```

```
    );
```

```
printf("Avg TAT: %.6fms", avg-tat/n);
```

```
printf("Avg WT: %.6fms", avg-wt/n);
```

```
}
```

Q10:

Enter no of process : 5

Enter TQ : 2

Enter RT:

0

1

2

3

4

Enter BT:

5

3

1

2

3

RES Scheduling:

PID	RT	BT	CT	TAT	WT	RT
1	0	5	14	14	9	0
2	1	3	12	11	8	0
3	2	1	5	3	2	0
4	3	2	7	4	2	0
5	4	3	13	9	6	0

Agg TAT : 8.200 ms

Agg WT : 45.400 ms

Program -2

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```

printf("Name: Nikhil HR      USN:1WA24CS187\n\n"); int n, i;
printf("Enter number of processes: ");
scanf("%d", &n);

int at[n], bt[n], rt[n], ct[n], tat[n], wt[n], q[n];

printf("Enter Arrival Time:\n");
for(i = 0; i < n; i++)
scanf("%d", &at[i]);

printf("Enter Burst Time:\n");
for(i = 0; i < n; i++)
{ scanf("%d", &bt[i]);
  rt[i] = bt[i];
}

printf("Enter Queue (1 = RR, 2 = FCFS):\n");
for(i = 0; i < n; i++)
  scanf("%d", &q[i]);

int tq; printf("Enter Time
Quantum: "); scanf("%d", &tq);

int t = 0, completed = 0;

printf("\nGantt Chart:\n| ");

while(completed < n)
{ int found = 0;

  for(i = 0; i < n; i++)

```

```

{ if(q[i] == 1 && rt[i] > 0 && at[i] <= t)
    { int exec = (rt[i] > tq) ? tq : rt[i];

        for(int k = 0; k < exec; k++)
        { printf("P%d ", i+1);
          t++; }

        rt[i] -= exec;
        if(rt[i] == 0)
        { ct[i] = t;
          completed++; }

        found = 1;
    }
}

if(!found)
{ for(i = 0; i < n; i++)
    { if(q[i] == 2 && rt[i] > 0 && at[i] <= t)
        {

            while(rt[i] > 0)
            {
                // check if any Q1 arrives → break
                int interrupt = 0;
                for(int j = 0; j < n;
                    j++)
                { if(q[j] == 1 && rt[j] > 0 && at[j] <= t)
                    { interrupt = 1;
                      break;
                    }
                }
            }
        }
    }
}

```

```

    }
    if(interrupt) break;

    printf("P%d ", i+1);
    rt[i]--;
; t++; }

if(rt[i] == 0)
{ ct[i] = t;
completed++; }

found = 1;
}
}
}

// CPU idle
if(!found)
{
    printf("Idle ");
    t++;
}
}

printf("\n");

// Calculate TAT & WT
for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
}

```

```

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++)
{ printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

Name: Nikhil HR USN:1WA24CS187

Enter number of processes: 4

Enter Arrival Time:

0 0 0 10

Enter Burst Time:

4 3 8 5

Enter Queue (1 = RR, 2 = FCFS):

1 1 2 1

Enter Time Quantum: 2

Gantt Chart:

| P1 P1 P2 P2 P1 P1 P2 P3 P3 P3 P4 P4 P4 P4 P4 P3 P3 P3 |

Process	AT	BT	CT	TAT	WT
P1	0	4	6	6	2
P2	0	3	7	7	4
P3	0	8	20	20	12
P4	10	5	15	5	0

PS C:\Users\User\Documents\LAB_REPORT> █

Prog-8

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int n1, n2;
```

```
    int i;
```

```
    printf("Enter no. of process : ");
```

```
    scanf("%d", &n1);
```

```
    printf("Enter priority : ");
```

```
    scanf("%d", &n2);
```

```
    int b1[20], b2[20];
```

```
    printf("Enter burst times : ");
```

```
    for (int i=0; i<n1; i++)
```

```
        scanf("%d", &b1[i]);
```

```
        scanf("%d", &b2[i]);
```

```
    printf("Enter the burst times for queue 2 : ");
```

```
    for (i=0; i<n2; i++)
```

```
        printf
```

```
        scanf("%d", &b2[i]);
```

```
        scanf("%d", &b2[i]);
```

```
    int time=0;
```

```
    printf("Execution time : ");
```

```
    for (int i=0; i<n1; i++)
```

```
        printf("t = %d to t = %d : Q1 + P = %d",
```

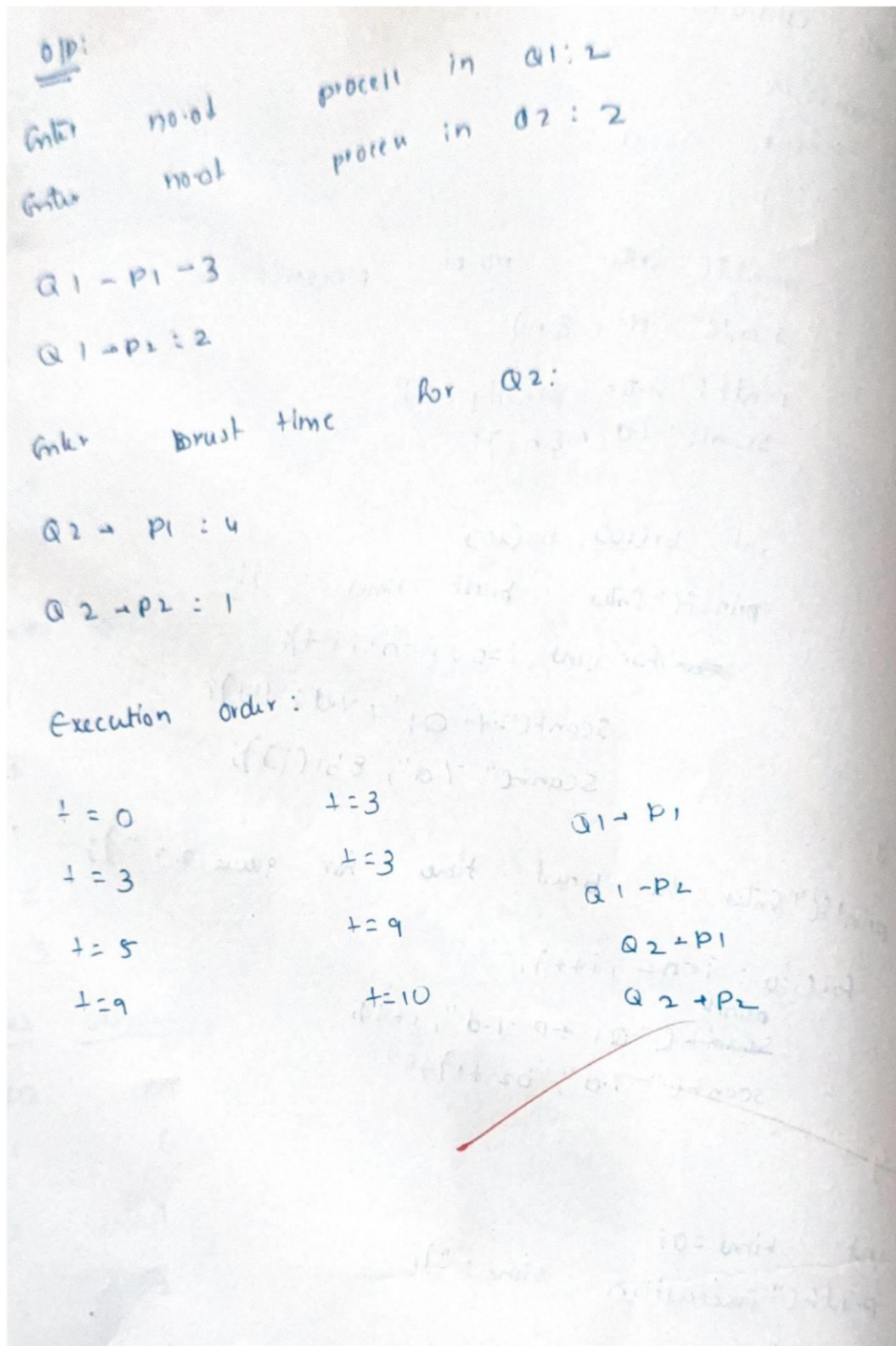
```
            time, time + b1[i], i+1);
```

```
    for (i=0; i<n2; i++)
```

```
        printf("t = %d to t = %d : Q2 : P = %d, time.
```

```
            time, time + b2[i], i+1);
```

```
            time + b2[i];
```

Program -3

Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one)

- Rate- Monotonic
- Earliest-deadline First
- Proportional scheduling

RATE – MONOTONIC

CODE:

```
#include <stdio.h>
#include<math.h>

int main()
{
printf("Name: Nikhil HR\nUSN:1WA24CS187\n\n");
    int n, i, j, t = 0, lcm = 1;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int burst[n], period[n], remaining[n];

    printf("Enter CPU burst times:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &burst[i]);

    printf("Enter time periods:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &period[i]);
    for(i = 0; i < n; i++)
    { int max = (lcm > period[i]) ? lcm : period[i];
        while(1)
        { if(max % lcm == 0 && max % period[i] == 0)
            { lcm = max;
                break;
            }
            max++;
        }
    }
}
```

```

printf("\nLCM=%d\n", lcm);

printf("\nRate Monotonic Scheduling:\n");
printf("PID\tBurst\tPeriod\n"); for(i = 0; i
< n; i++)
    printf("%d\t%d\t%d\n", i+1, burst[i], period[i]);

float util = 0; for(i
= 0; i < n; i++)
    util += (float)burst[i] / period[i];

float bound = n * (pow(2, (float)1/n) - 1);

printf("\n%f <= %f => %s\n", util, bound,
    (util <= bound) ? "true" : "false");
for(i = 0; i < n; i++)
    remaining[i] = 0;

printf("\nScheduling occurs for %d ms\n\n", lcm);

for(t = 0; t < lcm; t++)
{

    for(i = 0; i < n; i++)
    { if(t % period[i] == 0)
        remaining[i] = burst[i];
    }
}

```

```

int min = -1; for(i =
0; i < n; i++)
{ if(remaining[i] > 0)
    { if(min == -1 || period[i] < period[min])
        min = i;
    }
}

if(min != -1)
{ printf("%dms onwards: Process %d running\n", t, min+1);
  remaining[min]--;
}
else
{ printf("%dms onwards: CPU is idle\n", t);
} }

return 0;
}

```

Result:

```

Enter CPU burst times:
20 35
Enter time periods:
50 100
LCM=100

Rate Monotonic Scheduling:
PID    Burst    Period
1       20       50
2       35      100

0.750000 <= 0.828427 => true

Scheduling occurs for 100 ms

0ms onwards: Process 1 running
1ms onwards: Process 1 running
2ms onwards: Process 1 running
3ms onwards: Process 1 running
4ms onwards: Process 1 running
5ms onwards: Process 1 running
6ms onwards: Process 1 running
7ms onwards: Process 1 running
8ms onwards: Process 1 running
9ms onwards: Process 1 running
10ms onwards: Process 1 running
11ms onwards: Process 1 running
12ms onwards: Process 1 running
13ms onwards: Process 1 running
14ms onwards: Process 1 running
15ms onwards: Process 1 running
16ms onwards: Process 1 running
17ms onwards: Process 1 running
18ms onwards: Process 1 running
19ms onwards: Process 1 running
20ms onwards: Process 2 running
21ms onwards: Process 2 running
22ms onwards: Process 2 running
23ms onwards: Process 2 running
24ms onwards: Process 2 running
25ms onwards: Process 2 running
26ms onwards: Process 2 running
27ms onwards: Process 2 running
28ms onwards: Process 2 running
29ms onwards: Process 2 running
30ms onwards: Process 2 running

```

```
73ms onwards: Process 2 running
74ms onwards: Process 2 running
75ms onwards: CPU is idle
76ms onwards: CPU is idle
77ms onwards: CPU is idle
78ms onwards: CPU is idle
79ms onwards: CPU is idle
80ms onwards: CPU is idle
81ms onwards: CPU is idle
82ms onwards: CPU is idle
83ms onwards: CPU is idle
84ms onwards: CPU is idle
85ms onwards: CPU is idle
86ms onwards: CPU is idle
87ms onwards: CPU is idle
88ms onwards: CPU is idle
89ms onwards: CPU is idle
90ms onwards: CPU is idle
91ms onwards: CPU is idle
92ms onwards: CPU is idle
93ms onwards: CPU is idle
94ms onwards: CPU is idle
95ms onwards: CPU is idle
96ms onwards: CPU is idle
97ms onwards: CPU is idle
98ms onwards: CPU is idle
99ms onwards: CPU is idle
```

25 C:\Users\Hsara\Documents\LAB-REPORT>

Rate Monotonic Scheduling

```
#include <stdio.h>
```

```
int gcd(int a, int b) {
```

```
    while (b != 0) {
```

```
        int t = b;
```

```
        b = a % b;
```

```
        a = t;
```

```
    }
```

```
    return a;
```

```
}
```

```
int sum(int a, int b) {
```

```
    return (a / gcd(a, b) * b);
```

```
}
```

```
int main() {
```

```
    printf("Enter no. of tasks: ");
```

```
    scanf("%d", &n);
```

```
    int t[20], p[20], idx[20];
```

```
    int i, j;
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("Enter comp time c: ", i+1);
```

```
        scanf("%d", &t[i]);
```

```
        printf("Enter period P: ", i+1);
```

```
        scanf("%d", &p[i]);
```

```
        idx[i] = i;
```

```
    }
```

```

for (i=0; i < n-1; i++) {
    for (j=i+1; j < n; j++) {
        if (P[idx[i]] < P[idx[j]])
            int t = idx[i];
            idx[i] = idx[j];
            idx[j] = t;
    }
    int hyperperiod = P[0];
    for (i=1; i < n; i++) {
        hyperperiod = lcm + (hyperperiod);
    }
    if (sum == -1) {
        for (i=0; i < n; i++) {
            int t = idx[i];
            printf("Idle");
        }
        return 0;
    }
}

```

o/p:

PID
1
2

Brust
20
35

Period
50
100

Scheduling	occurs for	100ms
0	P1	
20	P2	
50	P1	
70	P2	
75	CPU idle	

EARLIEST DEADLINE FIRST

CODE:

```
#include <stdio.h>
#include <limits.h>

int main() {
    printf("Name: Nikhil HR\nUSN:1WA24CS187\n\n");
    int n, i, time, hyperperiod = 20;

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    int C[n], D[n], T[n];
    int remaining[n];
    int next_arrival[n];
    int abs_deadline[n];

    for(i = 0; i < n; i++) {
        printf("Enter Capacity, Deadline, Period for Task %d: ", i + 1);
        scanf("%d %d %d", &C[i], &D[i], &T[i]);

        remaining[i] = 0;
        next_arrival[i] = 0;
        abs_deadline[i] = 0;
    }

    printf("\n--- EDF Scheduling (Periodic Tasks) ---\n");

    for(time = 0; time < hyperperiod; time++) {
```



```

for(i = 0; i < n; i++) { if(time
    == next_arrival[i]) {
        remaining[i] = C[i];
        abs_deadline[i] = time + D[i];
        next_arrival[i] += T[i];
    }
}

int selected = -1; int
min_deadline = INT_MAX;

for(i = 0; i < n; i++) {
    if(remaining[i] > 0 && abs_deadline[i] < min_deadline) {
        min_deadline = abs_deadline[i];
        selected = i;
    }
}

if(selected == -1) {
    printf("%d ms : CPU is idle.\n", time);
} else { printf("%d ms : Task %d is running.\n", time,
    selected + 1); remaining[selected]--;
}

return 0;
}

```

Result:

Name: Nikhil HR
USN: 1WA24CS187

Enter number of tasks:
3

Enter Capacity, Deadline, Period for Task 1: 3 7 20
Enter Capacity, Deadline, Period for Task 2: 2 4 5
Enter Capacity, Deadline, Period for Task 3: 2 8 10

EDF Scheduling (Periodic Tasks) ---

0 ms : Task 2 is running.
1 ms : Task 2 is running.
2 ms : Task 1 is running.
3 ms : Task 1 is running.
4 ms : Task 3 is running.
5 ms : Task 3 is running.
6 ms : Task 3 is running.
7 ms : Task 2 is running.
8 ms : Task 2 is running.
9 ms : CPU is idle.
10 ms : Task 2 is running.
11 ms : Task 2 is running.
12 ms : Task 3 is running.
13 ms : Task 3 is running.
14 ms : CPU is idle.
15 ms : Task 1 is running.
16 ms : Task 2 is running.
17 ms : CPU is idle.
18 ms : CPU is idle.
19 ms : CPU is idle.

Earliest deadline first

```
#include <stdio.h>
```

```
#define INF 100000
```

```
int main()
```

```
{  
    int n;
```

```
    printf("Enter process: ");
```

```
    scanf("%d", &n);
```

```
    int ar[50], br[50], dead[50];
```

```
    int rem[50];
```

```
    int comp = 0;
```

```
    int time = 0;
```

```
    for (int i = 0; i < n; i++)
```

```
    {  
        printf("Enter AT, BT, deadline for P%d: ", i+1);
```

```
        scanf("%d %d %d", &ar[i], &br[i], &dead[i]);
```

```
        rem[i] = br[i];
```

```
    }
```

```
    printf("Execution timeline: ");
```

```
    while (comp < n)
```

```
    {  
        int min_deadline = INF;
```

```
        int sel = -1;
```

```
        for (int i = 0; i < n; i++)
```

```
        {  
            if (ar[i] <= time && rem[i] > 0)
```

```
            {  
                if (dead[i] < min)
```

```
                {  
                    min = dead[i];
```

```
                    sel = i;
```

```
                }
```

```
            }
```

```
        }
```

```

if (sel == -1) {
    printf("%d ms : CPU is idle", time);
    time++;
} else {
    printf("%d ms : Task %d is running", time,
           selected + 1);
    rem[sel]--;
    time++;
    if (rem[sel] == 0) {
        comp++;
    }
}
return 0;

```

O/P:

Enter process = 3

Enter AT, BT, dead for P1: 20 30 10

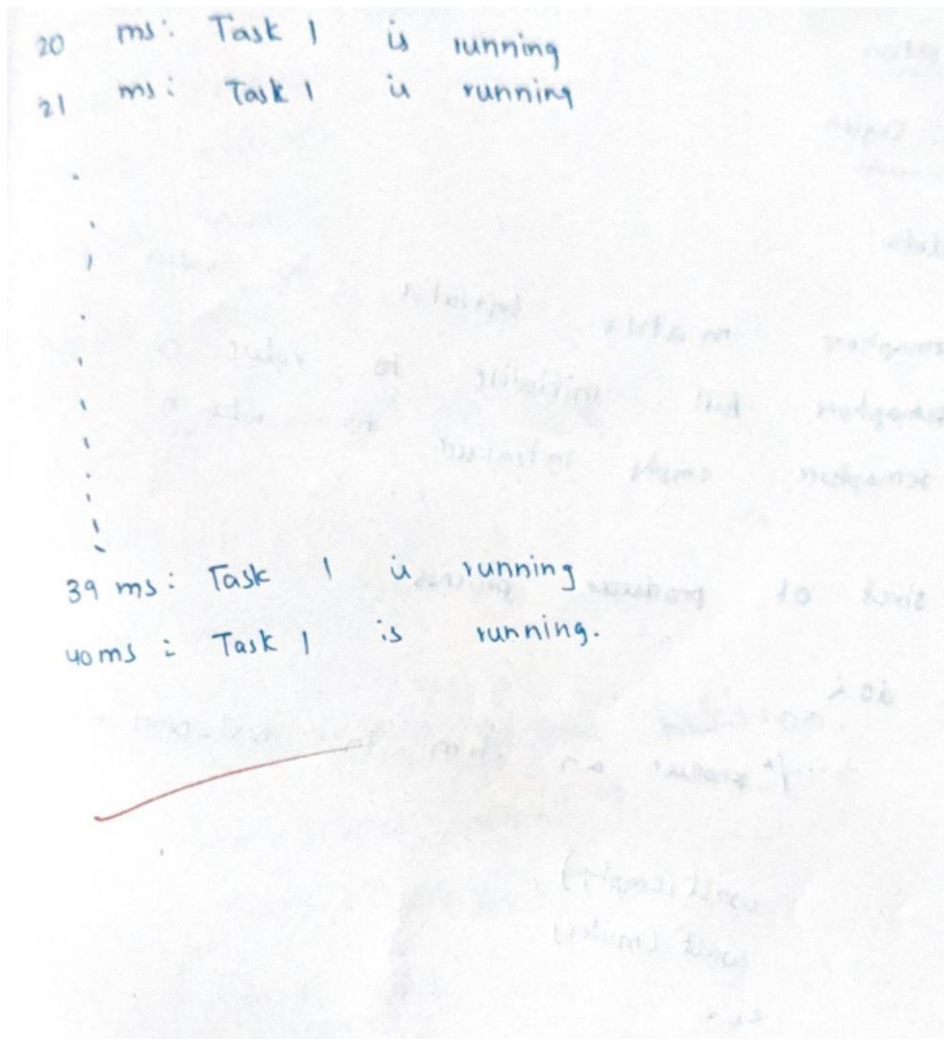
Enter AT, BT, dead for P2: 30 40 60

Enter AT, BT, dead for P3: 50 70 30

Exec timeline:

0 ms: CPU idle

1 ms: CPU idle



PROPORTIONAL SHARE SCHEDULING

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
typedef struct {
    char
    name[5]; int
```

```

tickets; }
Process;

int main() {
printf("Name: Nikhil HR\nUSN:1WA24CS187\n");
int n, total_tickets = 0;
float total_T = 0.0;

printf("Enter the number of Processes:
"); scanf("%d", &n); Process p[n];
srand(time(NULL));

for (int i = 0; i < n; i++) {
printf("\nProcess %d:\n", i + 1);
sprintf(p[i].name, "P%d", i + 1);
printf("Tickets: "); scanf("%d",
&p[i].tickets); total_tickets +=
p[i].tickets; total_T +=
p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---
\n"); printf("Enter the Time Period for scheduling:
"); int m; scanf("%d", &m);

for (int i = 0; i < m; i++) { int winning_ticket =
rand() % total_tickets + 1; int
accumulated_tickets = 0; int winner_index;

for (int j = 0; j < n; j++) {
accumulated_tickets += p[j].tickets; if
(winning_ticket <= accumulated_tickets) {

```

```

        winner_index = j;
        break;
    }
}

printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
        p[winner_index].name);
}

for (int i = 0; i < n; i++) {
    printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
            ((p[i].tickets / total_T) * 100));
}

return 0;
}
Result:

```

```
Name: Nikhil HR
USN: 1MA24CS187
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 35, Winner: P3
Tickets picked: 52, Winner: P3
Tickets picked: 9, Winner: P1
Tickets picked: 15, Winner: P2
Tickets picked: 7, Winner: P1

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.
PS C:\Users\User\Documents\LAB_REPORT> █
```


Proportional Share Scheduling

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

typedef struct {

char name[5]; int tic;

Process;

int main() {

int n, total-tic = 0; float total-t = 0.0;

printf("Enter process: ");

scanf("%d", &n);

process p[n];

srand(time(NULL));

for (int i = 0; i < n; i++) {

printf("Process " "%d", i+1);

sprintf(p[i].name, "P%d", i+1);

printf("Tickets: ");

scanf("%d", &p[i].tickets);

total-tic += p[i].tic;

total-T += p[i].tickets;

}

printf("Proportional share scheduln: ");

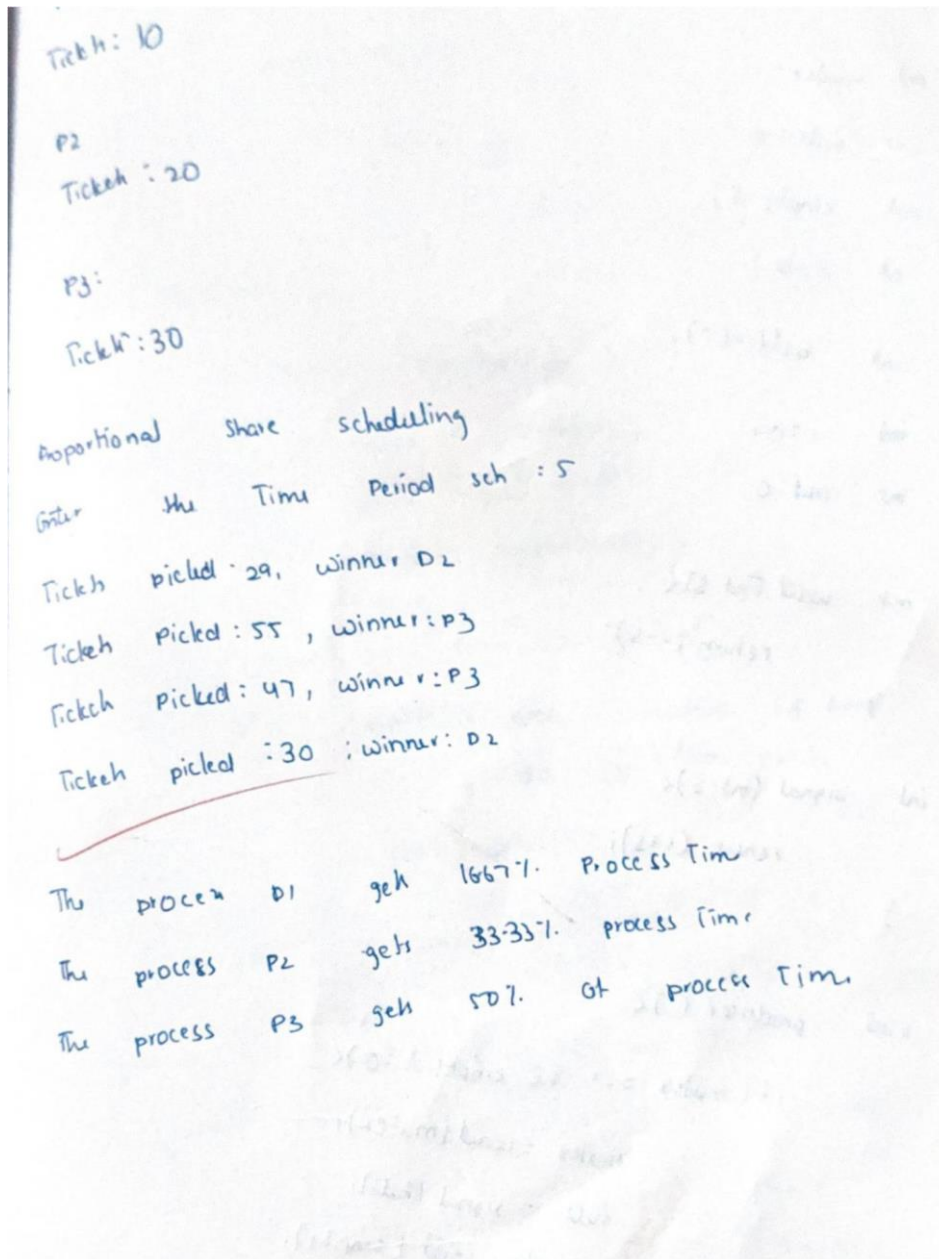
printf("Enter Time period: ");

int m;

```

srand (1.0 * time(0));
for (int i=0; i<n; i++) {
    int win = rand() % (total-tick + 1);
    int accumulated = tick = 0;
    int winner;
    for (int j=0; j<n; j++) {
        acc += p[i] * tick;
        if (win == acc - tick) {
            winner-index = j;
            break;
        }
    }
    printf("Ticket picked: %d, winner %d, winner-tick\n",
        p[winner-index].name);
}
for (int i=0; i<n; i++) {
    printf("The Process %s gets %.02f\n",
        p[i].name,
        ((p[i].tickets / total) * 100));
}
return 0;
}

```



Program -4

Question:

Write a C program to simulate: (Any one)

- Producer-Consumer problem using semaphores.
- Dining-Philosopher's problem

PRODUCER – CONSUMER

CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int
buffer[BUFFER_SIZE];
int top = -1; int item = 0;

sem_t empty, full, mutex;

void produce() {
    sem_wait(&empty);
    sem_wait(&mutex);

    item++; top++;
    buffer[top] = item;

    printf("Producer has produced: Item %d\n", item);
    sem_post(&mutex); sem_post(&full);
}

void consume() {
    sem_wait(&full);
    sem_wait(&mutex);

    int consumed = buffer[top];
    printf("Consumer has consumed: Item %d\n", consumed);

    top--;

```

```

sem_post(&mutex);
sem_post(&empty);
}

int main() {
printf("Name: Nikhil HR\nUSN:1WA24CS187\n\n");
int choice;

sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full, 0, 0);
sem_init(&mutex, 0, 1);

while (1) {
printf("\nEnter 1. Producer 2. Consumer 3.
Exit\n"); printf("Enter your choice: "); scanf("%d",
&choice);

switch (choice) {
case 1:
if (top == BUFFER_SIZE - 1) {
printf("Buffer is full!\n");
} else {
produce(); }
break;

case 2:
if (top == -1) {
printf("Buffer is empty!\n");
} else {
consume(); }
break;

```

```
    case 3:
        printf("Exiting...\n");
        exit(0);

    default:
        printf("Invalid choice!\n");
    }
}

return 0;
}
```

Result:

USN: 1WA24CS187

Enter 1. Producer 2. Consumer 3. Exit

Enter your choice: 1

Producer has produced: Item 1

Enter 1. Producer 2. Consumer 3. Exit

Enter your choice: 1

Producer has produced: Item 2

Enter 1. Producer 2. Consumer 3. Exit

Enter your choice: 2

Consumer has consumed: Item 2

Enter 1. Producer 2. Consumer 3. Exit

Enter your choice: 2

Consumer has consumed: Item 1

Enter 1. Producer 2. Consumer 3. Exit

Enter your choice: 2

Buffer is empty!

Enter 1. Producer 2. Consumer 3. Exit

Enter your choice: 3

Exiting...

7 Producer - consumer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int mutex = 1;
```

```
int full = 0;
```

```
int empty = 5;
```

```
int x = 0;
```

```
int buffer[5];
```

```
int i = 0;
```

```
int out = 0;
```

```
int wait (int s) {  
    return (--s);
```

```
}
```

```
int signal (int s) {  
    return (++s);
```

```
}
```

```
void producer () {
```

```
    if (mutex == 1 && empty != 0) {
```

```
        mutex = wait (mutex);
```

```
        full = signal (full);
```

```
        empty = wait (empty);
```

```
        x++;
```

```
        buffer[in] = x;
```

```
        printf("Producer produces item %d at slot %d", x, in);
```



```

        in = (in+1) % 5;

        mutex = signal (mutex);

    } else {
        printf(" Buffer  ");
    }
}

void consumer () {
    if (mutex == 1 && full != 0) {
        mutex = wait (mutex);
        full = wait (full);
        empty = signal (empty);

        int item = buffer [out];

        printf(" Consumer consumes %d from\n slot %d ", item, out);

        out = (out+1) % 5;

        mutex = signal (mutex);

    } else {
        printf(" Buffer empty");
    }
}

int main () {
    int n;
    printf(" Producer Consumer Exit");
}

```

switch (n1 <

case 1:

producer();

break;

case 2:

consumer();

break;

case 3:

printf("Exit");

exit(0);

break;

default:

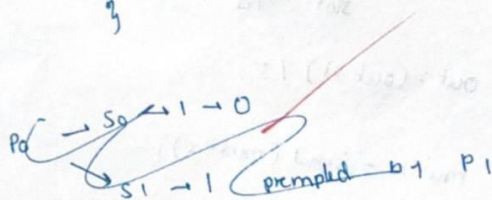
printf("Invalid");

}

}

return 0;

}



Op: ask

1. Producer

2. Consumer

3. Exit

Enter your choice: 1

Producer produces item 1 at slot 0

Enter your choice: 2

Consumer consumes item from slot 0

Producer produces item 2 at Slot 1

Enter your choice 2

Consumer consumes item 2 from slot 2

Enter your choice: 3

Exiting program.

DINING – PHILOSOPHER’S

CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher(void* num) {
    int id = *(int*)num; int
    left = id; int right = (id
    + 1) % N;

    for (int i = 0; i < 1; i++) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[left]); printf("Philosopher
            %d picked left fork %d\n", id, left);

            pthread_mutex_lock(&forks[right]); printf("Philosopher
            %d picked right fork %d\n", id, right);
        } else { pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked right fork %d\n", id,
            right);

            pthread_mutex_lock(&forks[left]);
```

```

        printf("Philosopher %d picked left fork %d\n", id, left);
    }

    printf("Philosopher %d is eating.\n", id);
    sleep(2);

    pthread_mutex_unlock(&forks[left]); pthread_mutex_unlock(&forks[right]);

    printf("Philosopher %d put down forks %d and %d\n", id, left, right);
}

return NULL;
}

int main() {
printf("Name: Nikhil HR\nUSN:1WA24CS187\n\n");
    int ids[N];

    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i; }

    for (int i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }
}

```

```
for (int i = 0; i < N; i++) {  
    pthread_mutex_destroy(&forks[i]);  
}  
  
return 0;  
}
```

Result:

Result:

Name: Nikhil HR

USN: 1WA24CS187

Philosopher 0 is thinking.

Philosopher 1 is thinking.

Philosopher 2 is thinking.

Philosopher 3 is thinking.

Philosopher 0 picked left fork 0

Philosopher 1 picked right fork 2

Philosopher 1 picked left fork 1

Philosopher 1 is eating.

Philosopher 3 picked right fork 4

Philosopher 3 picked left fork 3

Philosopher 3 is eating.

Philosopher 1 put down forks 1 and 2

Philosopher 0 picked right fork 1

Philosopher 0 is eating.

Philosopher 2 picked left fork 2

Philosopher 3 put down forks 3 and 4

Philosopher 2 picked right fork 3

Philosopher 2 is eating.

Philosopher 0 put down forks 0 and 1

Philosopher 2 put down forks 2 and 3

//code

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```
#define N 5
```

```
pthread_mutex_t forks[N];
pthread_t philosopher[N];
```

```
void * philosopher(void * num)
```

```
{
    int id = *(int *)num;
```

```
    int left = id;
```

```
    int right = (id+1) % N;
```

```
    while (1) {
```

```
        printf("philosopher %d\n", id);
```

```
        sleep(1);
```

```
        if (id % 2 == 0) {
```

```
            pthread_mutex_lock(&forks[left]);
```

```
            printf("philosopher %d left fork %d\n", id, left);
```

```
            pthread_mutex_lock(&forks[right]);
```

```
            printf("philosopher %d picked up %d\n",
```

```
                id, right);
```

```
        }
```

```
    }
```



```

pthread - mutex - lock (&forks [right]);
printf ("philosopher %d picked up right fork %d, id,
                                             right);

pthread - mutex - lock (&forks [left]);
printf ("philosopher %d picked up left fork %d,
                                             id, left);
}
printf ("philosopher %d is eating ", id);
sleep (2);

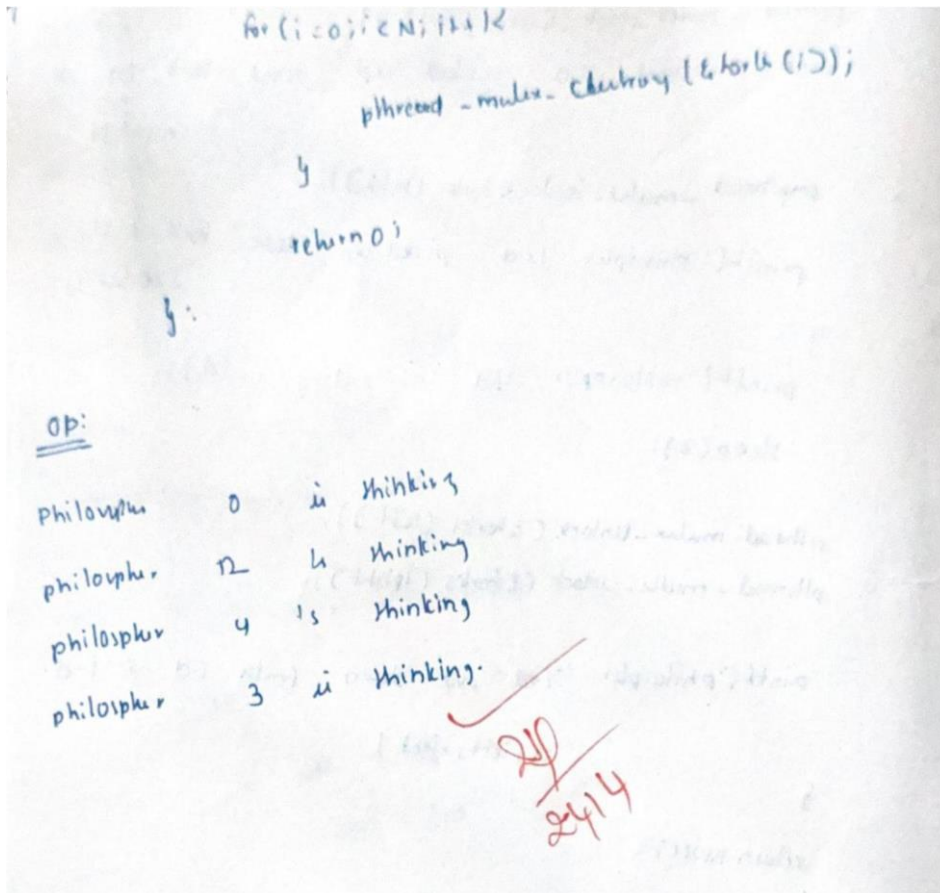
pthread - mutex - unlock (&forks [left]);
pthread - mutex - unlock (&forks [right]);

printf ("philosopher %d put down forks %d & %d.
                                             id, left, right);
}
return NULL;
}

int main () {
    int i;
    int ids [N];

    for (i=0; i<N; i++) {
        pthread - mutex - init (&forks [i], NULL);
        ids [i] = i;
    }
    for (i=0; i<N; i++) {
        pthread - create (&philosophers [i], NULL, philosopher,
                                             &ids [i]);
    }
    for (i=0; i<N; i++) {
        pthread - join (philosophers [i], NULL);
    }
}

```

Program -5

Question:

Write a C program to simulate: (Any one)

- Bankers' algorithm for the purpose of deadlock avoidance.
- Deadlock Detection

BANKER'S ALGORITHM

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
printf("Name: Nikhil HR \nUSN:1WA24CS187\n\n");
```

```
int n, m, i, j, k;
```

```

printf("Enter number of processes: ");
scanf("%d", &n);

printf("Enter number of resources: ");
scanf("%d", &m);

int alloc[n][m], max[n][m], need[n][m];
int avail[m];

printf("Enter Allocation Matrix:\n");
for(i = 0; i < n; i++)
{ for(j = 0; j < m; j++)
    { scanf("%d", &alloc[i][j]);
    }
}
printf("Enter Maximum Demand Matrix:\n");
for(i = 0; i < n; i++)
{ for(j = 0; j < m; j++)
    { scanf("%d", &max[i][j]);
    }
}

printf("Enter Available Resources:\n");
for(i = 0; i < m; i++)
{ scanf("%d", &avail[i]);
}

// Calculate Need Matrix
for(i = 0; i < n; i++)
{ for(j = 0; j < m; j++)
    { need[i][j] = max[i][j] - alloc[i][j];
    }
}

```

```
    }  
}
```

```
int finish[n], safeSeq[n];  
int work[m];
```

```
for(i = 0; i < n; i++)  
    finish[i] = 0;
```

```
for(i = 0; i < m; i++)  
    work[i] = avail[i];
```

```
int count = 0;
```

```
while(count < n)  
{ int found = 0;
```

```
    for(i = 0; i < n; i++)  
    { if(finish[i] == 0)  
        { int possible = 1;
```

```
            for(j = 0; j < m; j++)  
            { if(need[i][j] > work[j])  
                { possible = 0;  
                  break;  
                }  
            }  
        }
```

```
        if(possible)  
        { for(k = 0; k < m; k++)  
            { work[k] += alloc[i][k];  
            }  
        }
```

```

        safeSeq[count] = i;
        count++;

        finish[i] = 1;
        found = 1;
    }
}

if(found == 0)
{ printf("System is not in a safe state.\n");
  return 0;
}

printf("\nSystem is in a safe state.\n");
printf("Safe sequence is: ");

for(i = 0; i < n; i++)
{ printf("P%d", safeSeq[i]);

    if(i != n - 1)
        printf(" -> ");
}

printf("\n");

return 0;
}
Result:

```

Name: Nikhil HR

USN: 1WA24CS187

Enter number of processes: 5

Enter number of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Demand Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available Resources:

3 3 2

System is in a safe state.

Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2

Banks algorithm

#include <stdio.h>

int main ()

int n, m, i, j, k;

printf("Banks Algorithm");

printf("Enter no of process: ");

scanf("%d", &n);

printf("Enter no of resources: ");

scanf("%d", &m);

int alloc[n][m], max[n][m], need[n][m];

int avail[m];

printf("Enter Alloc matrix: ");

for (int i = 0; i < n; i++)

for (int j = 0; j < m; j++)

scanf("%d", &alloc[i][j]);

printf("Enter maximum Demand Matrix:");

for (i = 0; i < n; i++)

for (j = 0; j < m; j++)

scanf("%d", &max[i][j]);

}

```

printf("Enter Availabl  

for (i=0; i<m; i++)<
    scanf ("%d", &avail[i]);
}

for (i=0; i<n; i++)<
    for (j=0; j<m; j++)<
        need[i][j] = max(i)[j] - alloc(i)[j];
    }
}

```

```

int finish(n), safe seq(n), work[m];

```

```

for (i=0; i<n; i++)
    finish[i] = 0;

```

```

for (i=0; i<m; i++)
    work[i] = avail[i];

```

```

int count = 0;

```

```

while (count < n)

```

```

    int found = 0;

```

```

    for (i=0; i<n; i++)

```

```

        if (finish[i] == 0)

```

```

            int pass = 1;

```

```

            for (j=0; j<m; j++)

```

```

                if (need[i][j] > work[j])
                    pass = 0;

```



```

if (possible) {
    for (k=0; k<m; k++)
        work[k] = alloc[i][k];

    safeSeq[count++] = i;
    finish[i] = 1;
    found = 1;
}
}

if (found == 0)
    break;
}

if (count == n)
    printf("system in safe state");
    printf("Safe seq is: ");

    for (i=0; i<n; i++)
        printf("%d", safeSeq[i]);

        if (i != n-1)
            printf(" ");
    }
    printf("\n");
}
else {
    printf("System is NOT in a safe state.");
}

return 0;
}

```


Q/p

Enter no of process : 5

Enter no of resources : 3

Enter allocation matrix:

0	1	0
2	0	0
3	0	2
2	1	1
0	0	2

Enter max demand matrix:

7	5	3
---	---	---

3	2	2
---	---	---

9	0	2
---	---	---

2	2	2
---	---	---

4	3	3
---	---	---

Enter Avail Resource:

3	3	2
---	---	---

System is in safe state.

Safe sequence: $P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$

DEADLOCK DETECTION

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: Nikhil HR\nUSN:1WA24CS187\n\n");
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], req[n][m]; int
    avail[m], finish[n], safe[n];

    printf("Enter allocation matrix:\n");
    for(i = 0; i < n; i++)
    { printf("Process %d: ", i);
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    }

    printf("Enter request matrix:\n");
    for(i = 0; i < n; i++)
    { printf("Process %d: ", i);
        for(j = 0; j < m; j++)
            scanf("%d", &req[i][j]);
    }
```

```

printf("Enter available resources: ");
for(i = 0; i < m; i++)
    scanf("%d", &avail[i]);

for(i = 0; i < n; i++)
    finish[i] = 0;

int count = 0;

while(count < n)
{ int found = 0;

    for(i = 0; i < n; i++)
    { if(finish[i] == 0)
        { int possible = 1;

            for(j = 0; j < m; j++)
            {
                if(req[i][j] > avail[j])
                { possible = 0;
                    break;
                }
            }

            if(possible)
            { for(k = 0; k < m; k++)
                avail[k] += alloc[i][k];

                safe[count] = i;
                count++;
                finish[i] = 1;
                found = 1;
            }
        }
    }
}

```

```

        }
    }
}

if(found == 0)
    break;
}

if(count == n)
{ printf("\nSystem is in safe state.\n");
  printf("Safe Sequence is: ");

  for(i = 0; i < n; i++)
      printf("P%d ", safe[i]);
}
else
{ printf("\nDeadlock detected.\n");
}

return 0;
}

```

```

printf("Enter Available\n");
for (i=0; i<m; i++) {
    scanf("%d", &avail[i]);
}

for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int Finish(n, safe_seq(n), work[m]);

for (i=0; i<n; i++)
    finish[i] = 0;

for (i=0; i<m; i++)
    work[i] = avail[i];

int count = 0;
while (count < n) {
    int found = 0;
    for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            int poss = 1;
            for (j=0; j<m; j++) {
                if (need[i][j] > work[j])
                    poss = 0;
            }
            if (poss) {
                for (j=0; j<m; j++)
                    work[j] -= need[i][j];
                finish[i] = 1;
                count++;
            }
        }
    }
}

```

Deadlock detection also

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int n, m, i, j, t;
```

```
    printf("Deadlock detection");
```

```
    printf("Enter no. of process: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the no. of resources: ");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m];
```

```
    int req[n][m];
```

```
    int avail[n][m];
```

```
    printf("Enter allocation matrix: ");
```

```
    for (int i = 0; i < n; i++)
```

```
    {  
        printf("Process %d: ", i);
```

```
        for (j = 0; j < m; j++)
```

```
        {  
            scanf("%d", &alloc[i][j]);
```

```
        }  
    }
```

```
    printf("Enter the req matrix: ");
```

```
    for (i = 0; i < n; i++)
```

```
    {  
        printf("Process %d: ", i);
```

```
        for (j = 0; j < m; j++)
```

```
        {  
            scanf("%d", &req[i][j]);
```

```
        }  
    }
```

```

for (i=0; i<m; i++) {
    scanf("%d", &avail[i]);
}

int work[m];
int finish[n];
int selectseq(n);

for (i=0; i<m; i++)
    work[i] = avail[i];

for (i=0; i<n; i++) {
    int zero = 1;

    for (j=0; j<m; j++) {
        if (alloc[i][j] != 0) {
            zero = 0;
            break;
        }
    }

    if (zero)
        finish[i] = 1;
    else
        finish[i] = 0;
}

int count = 0;
while (count < n) {
    int found = 0;
    for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            int possi = 1;

```



```

for (j=0; j < m; j++)
    if (req[i][j] > work[j])
        poss = 0;
        break;
}

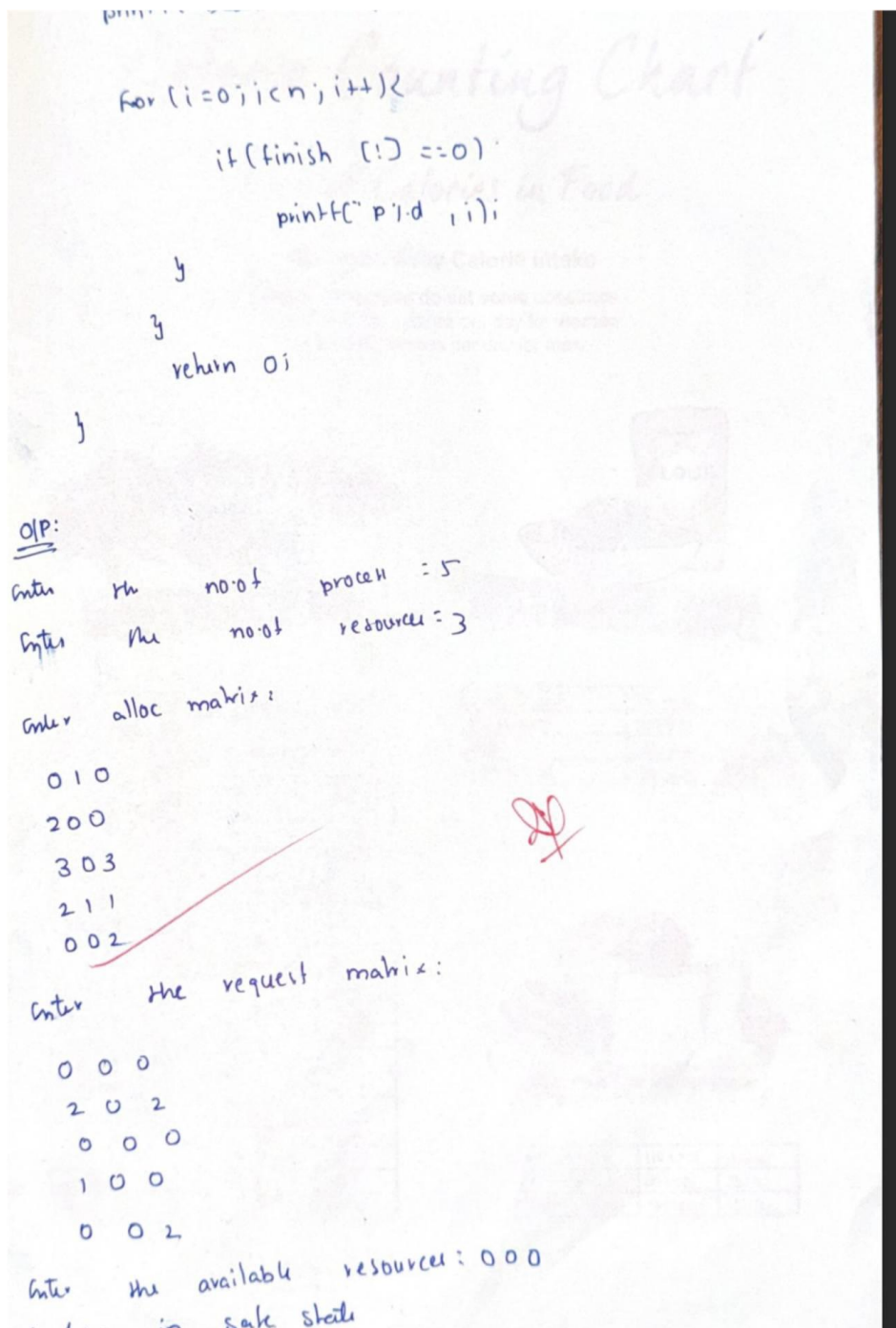
if (poss)
    for (k=0; k < m; k++)
        work[k] += alloc[i][k];
    safe_seq[count++] = i;
    finish[i] = 1;
    found = 1;
}

if (found == 0)
    break;
}

for (i=0; i < n; i++)
    if (finish[i] == 0)
        deadlock = 1;
        break;
}

if (deadlock == 0)
    printf("sys in safe seq : ");
    printf("safe seq is : ");
    for (i=0; i < count; i++)
        printf("pid, safe seq[i]");
    printf("\n");
}
else

```

Program -6

Question:

Write a C program to simulate the following contiguous memory allocation techniques. (Any one)

- Worst-fit
- Best-fit
- First-fit

CODE:

```
#include <stdio.h>

void firstFit(int blocks[], int m, int processes[], int n) {
    int alloc[n], used[m];
    for (int i = 0; i < m;
        i++) used[i] = 0;
    for (int i = 0; i < n; i++)
        alloc[i] = -1;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (!used[j] && blocks[j] >= processes[i]) {
                alloc[i] = j;
                used[j] = 1;
                break;
            }

    printf("\n--- First Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processes[i]);
        if (alloc[i] != -1)
            printf("%d\n", alloc[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blocks[], int m, int processes[], int n) {
```

```

int alloc[n], used[m];
for (int i = 0; i < m;
i++) used[i] = 0;
for (int i = 0; i < n; i++)
    alloc[i] = -1;

for (int i = 0; i < n; i++) {
    int best = -1; for (int j =
0; j < m; j++)
        if (!used[j] && blocks[j] >= processes[i])
            if (best == -1 || blocks[j] < blocks[best])
                best = j;
    if (best != -1) {
        alloc[i] = best;
        used[best] = 1;
    }
}

printf("\n--- Best Fit ---\n"); printf("Process
No.\tProcess Size\tBlock No.\n"); for (int i = 0; i
< n; i++) {
    printf("%d\t%d\t", i + 1, processes[i]);
    if (alloc[i] != -1)
        printf("%d\n", alloc[i] + 1);
    else printf("Not
        Allocated\n");
}
}

void worstFit(int blocks[], int m, int processes[], int n) {

```

```

int alloc[n], used[m];
for (int i = 0; i < m;
i++) used[i] = 0;
for (int i = 0; i < n; i++)
    alloc[i] = -1;

for (int i = 0; i < n; i++) {
    int worst = -1; for (int j
    = 0; j < m; j++)
        if (!used[j] && blocks[j] >= processes[i])
            if (worst == -1 || blocks[j] > blocks[worst])
                worst = j;
    if (worst != -1) {
        alloc[i] = worst;
        used[worst] = 1;
    }
}

printf("\n--- Worst Fit ---\n"); printf("Process
No.\tProcess Size\tBlock No.\n"); for (int i = 0; i
< n; i++) {
    printf("%d\t%d\t", i + 1, processes[i]);
    if (alloc[i] != -1)
        printf("%d\n", alloc[i] + 1);
    else printf("Not
        Allocated\n");
}
}

int main() {

    printf("Name: Nikhil HR\nUSN:1WA24CS187\n\n");

```

```

int m, n;

printf("Enter number of memory blocks:
"); scanf("%d", &m); int blocks[m];
printf("Enter sizes of %d memory blocks:\n", m);
for (int i = 0; i < m; i++)
    scanf("%d", &blocks[i]);

printf("Enter number of processes:
"); scanf("%d", &n); int processes[n];
printf("Enter sizes of %d processes:\n", n);
for (int i = 0; i < n; i++)
    scanf("%d", &processes[i]);

firstFit(blocks, m, processes, n);
bestFit(blocks, m, processes, n);
worstFit(blocks, m, processes, n);

return 0;
}

```

```
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
2122 417 112 426
```

Process No.	Process Size	Block No.
-----	-----	-----
1	2122	Not Allocated
2	417	2
3	112	3
4	426	5

Process No.	Process Size	Block No.
-----	-----	-----
1	2122	Not Allocated
2	417	2
3	112	3
4	426	5

Process No.	Process Size	Block No.
-----	-----	-----
1	2122	Not Allocated
2	417	5
3	112	2
4	426	Not Allocated

600	312	600	512	600	512
300		300	212	300	
200		200		200	
700	812	700	526	700	212
400	212	400	312	400	312
	F		B		W

~~1412~~

```
#include <stdio.h>
```

```
int main()
```

```
{
    int nb, np, i, j;
```

```
    printf("Enter no. of memory blocks: ");
```

```
    scanf("%d", &nb);
```

```
    int block[20], b1[20], b2[20], b3[20];
```

```
    printf("Enter no. of memory blocks: ");
```

```
    scanf
```

```
    for (i=0; i<nb; i++)
```

```
    {
        scanf("%d", &block[i]);
```

```
        b1[i] = b2[i] = b3[i] = block[i];
```

```
    }
```

```
    printf("Enter no. of processes: ");
```

```
    scanf("%d", &np);
```

```

int process[20];
printf("Enter size of processes:");
for (i=0; i<np; i++)
    scanf("%d", &process[i]);

printf(" --- First Fit --- ");
printf("Process No\tProcess size\tBlockNo");
for (i=0; i<np; i++) {
    int allocated=0;
    for (j=0; j<nb; j++) {
        if (b1[j] >= process[i]) {
            printf("%d\t%d\t%d",
                i+1, process[i], j+1);
            b1[j] -= process[i];
            allocated = 1;
            break;
        }
    }
    if (!allocated)
        printf("%d\t%d\tNot Allocated",
            i+1, process[i]);
}

printf(" --- Worst Fit --- ");
printf("Process No\tProcess size\tBlock No");

for (i=0; i<np; i++) {
    int worst = -1;
    for (j=0; j<nb; j++) {

```



```

if (b3[j] >= process[i])
{
    if (worst == -1 || b2[j] > b2[worst])
    {
        worst = j;
    }
}

if (worst != -1)
{
    printf("%d %d %d", i+1, process[i],
           b3[worst] - process[i]);
    b3[worst] -= process[i];
}
else
{
    printf("%d %d Not allocated, it's",
           process[i]);
}

printf("Best Fit");
printf("Process No - Process Size ProcessBlock No");

for (i=0; i<nb; i++)
{
    int best = -1;
    for (j=0; j<nb; j++)
    {
        if (b2[j] >= process[i])
        {
            if (best == -1 || b2[j] < b2[best])
            {
                best = j;
            }
        }
    }
}

```

```

if (best != -1)
{
    printf("%d %d %d", i+1, process[i], best+1);
    best = process[i];
}
else
    printf("%d %d Not Allocated", i+1,
        process[i]);
}
return 0;
}

```

Enter no. of memory block is
Enter size of memory block.

400

700

200

300

600

Enter the no. of process = 4

Enter the size of process

212

512

312

526

First

P1 → Block 1

P2 → Block 2

P3 → Block 5

P4 → Not allowed

Best fit

P1 → Block 4

P2 → Block 5

P3 → Block 1

P4 → Block 2

Worst fit

P1 → Block 2

P2 → Block 5

P3 → Block 1

P4 → Not allocated

Program -7

Question:

Write a C program to simulate page replacement algorithms. (Any one)

- a) FIFO
- b) LRU
- c) Optimal

FIFO

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Name: Nikhil HR\nUSN: 1WA24CS187\n\n");
```

```
    int pages[20], frames[10]; int n, f,  
    i, j, k = 0, fault = 0, found;
```

```
    printf("Enter number of pages: ");  
    scanf("%d", &n);
```

```
    printf("Enter page reference string: ");  
    for(i = 0; i < n; i++) {  
        scanf("%d", &pages[i]);  
    }
```

```
    printf("Enter number of frames: ");  
    scanf("%d", &f);
```

```
    for(i = 0; i < f; i++) {  
        frames[i] = -1;  
    }  
    for(i = 0; i < n; i++) {
```

```

found = 0;

for(j = 0; j < f; j++) {
    if(frames[j] == pages[i]) {
        found = 1;
        break;
    }
}

if(found == 0) {
    frames[k] = pages[i];
    k = (k + 1) % f;
    fault++;
}

printf("\nPage %d -> ", pages[i]);

for(j = 0; j < f; j++) {
    if(frames[j] != -1)
        printf("%d ", frames[j]);
    else
        printf("- ");
}

printf("\n\nTotal Page Faults = %d\n", fault);

return 0;
}
Result:

```

Result

Name: Nikhil HR

USN: 1WA24CS187

Enter number of pages: 12

Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Enter number of frames: 2

Page 7 -> 7 -

Page 0 -> 7 0

Page 1 -> 1 0

Page 2 -> 1 2

Page 0 -> 0 2

Page 3 -> 3 2

Page 0 -> 0 3

Page 4 -> 4 3

Page 2 -> 4 2

Page 3 -> 3 2

Page 0 -> 3 0

Page 3 -> 3 0

Total Page Faults = 10

===== Execution Successful =====

FIFO

```
#include <stdio.h>
```

```
int main ()
```

```
    int pg, fr;
```

```
    int n, f, i, j;
```

```
    int pgf = 0;
```

```
    int next = 0;
```

```
    int found;
```

```
    printf("Enter no. of pg: ")
```

```
    scanf("%d", &n);
```

```
    printf("Enter no. of ref string: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter no. of frames: ");
```

```
    scanf("%d", &f);
```

```
    for (i = 0; i < f; i++)
```

```
        fr[i] = -1;
```

```
    }
```

```
    printf("FIFO: ")
```

```
    for (i = 0; i < n; i++)
```

```
        found = 0;
```

```
        for (j = 0; j < f; j++)
```

```
            if (frames[j] == page[i])
```

```
                found = 1;
```

```
                break;
```

```
        }
```

```

if (found == 0) {
    frames[next] = page[i];
    next = (next + 1) % f;
    pt++;
}
printf("Page %d ", page[i]);

for (j = 0; j < f; j++) {
    if (frames[j] != 1)
        printf("%d ", frames[j]);
    else
        printf(" ");
}
printf("\n");
}

printf("Total no. of faults %d, pt);

return 0;
}

```

O/P:

Enter no. of page: 12

Enter page ref string:

7 0 1 2 0 3 0 4 2 3 0 3

Enter no. of frames: 3.

P 7 $\rightarrow$ [7]

P 0 $\rightarrow$ [7 0]

P 1 $\rightarrow$ [7 0 1]

P 2 $\rightarrow$ [2 0 1]

P 0 $\rightarrow$ [2 0 1]

P 3 $\rightarrow$ [2 3 1]

P 0 $\rightarrow$ [2 3 0]

P 4 $\rightarrow$ [4 3 0]

P 2 $\rightarrow$ [4 2 0]

P 3 $\rightarrow$ [4 2 3]

P 0 $\rightarrow$ [0 2 3]

P 3 $\rightarrow$ [0 2 3]

Total page faults : 10

LRU

CODE:

```
#include <stdio.h>

int main() {

    printf("Name: Nikhil HR\nUSN: 1WA24CS187\n\n");

    int pages[20], frames[10], time[10]; int n,
    f, i, j, pos, fault = 0, count = 0, found;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string: ");
    for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++) {
        frames[i] = -1;
    }

    for(i = 0; i < n; i++) {

        found = 0;

        for(j = 0; j < f; j++) {
```

```

        if(frames[j] == pages[i]) {
            count++;
            time[j] = count;
            found = 1;
            break;
        }
    }

    if(found == 0) {

        pos = 0;

        for(j = 1; j < f; j++) {
            if(time[j] < time[pos]) {
                pos = j;
            }
        }

        frames[pos] =
pages[i]; count++;
time[pos] = count;
fault++; }

    printf("\nPage %d -> ", pages[i]);

    for(j = 0; j < f; j++) {
        if(frames[j] != -1)
            printf("%d ", frames[j]);
        else printf("-
");
    }
}

```

```

printf("\n\nTotal Page Faults = %d\n", fault);

return 0;
}

```

Result:

```

Result
-----
Name: Nikhil HR
USN: 1WA24CS187

Enter number of pages: 12
Enter page reference string: 1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

Page 1 -> 1 - -
Page 2 -> 1 2 -
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 4 1 3
Page 2 -> 4 1 2
Page 5 -> 5 1 2
Page 1 -> 5 1 2
Page 2 -> 5 1 2
Page 3 -> 5 3 2
Page 4 -> 5 3 4
Page 5 -> 5 3 4

Total Page Faults = 12

===== Execution Successful =====

```

OPTIMAL PAGE REPLACEMENT

CODE:

```

#include <stdio.h>

int main() {

    printf("Name: Nikhil HR\nUSN: 1WA24CS187\n\n");
}

```

```

int pages[20], frames[10]; int n, f,
i, j, k, pos, fault = 0, found;

printf("Enter number of pages: ");
scanf("%d", &n);

printf("Enter page reference string: ");
for(i = 0; i < n; i++) {
    scanf("%d", &pages[i]);
}

printf("Enter number of frames: ");
scanf("%d", &f);

for(i = 0; i < f; i++) {
    frames[i] = -1;
}

for(i = 0; i < n; i++) {

    found = 0;

    for(j = 0; j < f; j++) {
        if(frames[j] == pages[i]) {
            found = 1;
            break;
        }
    }

    if(found == 0) {

        pos = -1;
    }
}

```

```

for(j = 0; j < f; j++) {

    int future = 0;

    for(k = i + 1; k < n; k++) {
        if(frames[j] == pages[k]) {
            future = 1;
            break;
        }
    }

    if(future == 0)
    { pos = j;
      break;
    }
}

if(pos == -1)
    pos = 0;

frames[pos] = pages[i];
fault++;

}

printf("\nPage %d -> ", pages[i]);

for(j = 0; j < f; j++) {
    if(frames[j] != -1)
        printf("%d ", frames[j]);
    else
        printf("- ");
}

```

```

    }
}

printf("\n\nTotal Page Faults = %d\n", fault);

return 0;
}

```

Result:

```

Result
-----
Name: Nikhil HR
USN: 1WA24CS187

Enter number of pages: 12
Enter page reference string: 1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

Page 1 -> 1 - -
Page 2 -> 1 2 -
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 1 2 3
Page 2 -> 1 2 3
Page 5 -> 5 2 3
Page 1 -> 1 2 3
Page 2 -> 1 2 3
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 5 -> 5 2 3

Total Page Faults = 9

===== Execution Successful =====

```

```

#include <stdio.h>

int main() {
    int pg, fr;
    int n, f, i, j, k;
    int pgf = 0;
    int found, pos, forth, index;

    printf("Enter no. of pg: ");
    scanf("%d", &n);

    printf("Enter pg reference string:");
    for (i = 0; i < n; i++) {
        scanf("%d", &pg[i]);
    }

    printf("Enter no. of frames:");
    scanf("%d", &f);

    for (i = 0; i < f; i++) {
        frames[i] = -1;
    }

    printf("OPR ");

    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < f; j++) {
            if (frames[j] == pg[i]) {
                found = 1;
                break;
            }
        }
    }
}

```



```
if (found == 0) {
```

```
    pos = -1;
```

```
    for (j=0; j < f[j]; j++) {
```

```
        if (frames[j] == -1) {
```

```
            pos = j;
```

```
            break;
```

```
        }
```

```
    }
```

```
    if (pos != -1) {
```

```
        frames = page[i];
```

```
    } else {
```

```
        farthest = -1;
```

```
        index = -1;
```

```
        for (j=0; j < f[j]; j++) {
```

```
            int ff = 0;
```

```
            for (k=i+1; k < n; k++) {
```

```
                if (h[j] == pg[k]) {
```

```
                    if (k > farthest) {
```

```
                        farthest = k;
```

```
                        index = j;
```

```
                    }
```

```
                    found_hure = 1;
```

```
                    break;
```

```
                }
```

```
            }
```

```

        if (found future == 0) <
            index = j;
            break;
        }
    }

    frames[index] = pages[i];
    pageFault++;
}

printf("Page %d", pages[i]);

for (j=0; j < f; j++) <
    if (frames[j] != 1)
        printf("id %d", frames[j]);
    else
        printf(" ");
    if (j != f-1)
        printf(" ");
}

printf("Total pg fault : %d", pageFault);
}
OPR
LRU ←
for (i=0; i < pg; i++) <
    found = 0;
    for (j=0; j < fr; j++) <
        if (frames[j] == pages[i]) <
            found = 1;
        cnt++;
        time[j] = cnt;

```

O/p:

Enter the no of pages

```
if (!fault) {
```

```
    min = 0;
```

```
    for (j = 1; j < 3; j++) {
```

```
        if (time[j] < time[min])
```

```
            min = j;
```

```
    }
```

```
    frames[min] = page[i];
```

```
    cnt++;
```

```
    time[min] = cnt;
```

```
    fault++;
```

```
    printf("Page %d → [%d %d %d],
```

```
           page[i], fr[0], fr[1], fr[2]
```

```
    }
```

```
    printf("Total pf %d", fault);
```

```
}
```

O/p:

Enter no of pages: 12

Enter page reference string:

7 0 1 2 0 3 0 4 2 3 0 3

Enter no of frames: 3

LRU:

P 7 $\rightarrow$ [7]

P 0 $\rightarrow$ [7 0]

1 $\rightarrow$ [7 0 1]

2 $\rightarrow$ [0 1 2]

0 $\rightarrow$ [1 2 0]

3 $\rightarrow$ [2 0 3]

0 $\rightarrow$ [2 3 0]

4 $\rightarrow$ [3 0 4]

2 $\rightarrow$ [0 4 2]

3 $\rightarrow$ [4 2 3]

0 $\rightarrow$ [2 3 0]

3 $\rightarrow$ [2 0 3]

~~2~~

Total Page Faults (LRU): 9

- OPR $\rightarrow$

7 $\rightarrow$ [7]

0 $\rightarrow$ [7 0]

1 $\rightarrow$ [7 0 1]

2 $\rightarrow$ [2 0 1]

0 $\rightarrow$ [2 0 1]

3 $\rightarrow$ [2 0 3]

0 $\rightarrow$ [2 0 3]

4 $\rightarrow$ [2 4 3]

2 $\rightarrow$ [2 4 3]

3 $\rightarrow$ [2 4 3]

0 $\rightarrow$ [0 4 3]

3 $\rightarrow$ [0 4 3]

Total pf: 7

I N D E X

Name NIRHIL H R Std 100
 Roll No. Subject OS - lab obs School/College
 School/College Tel. No. Parents Tel. No.

Sl. No.	Date	Title	Page No.	Teacher Sign / Remarks
1	27/2/26	C program for finding smallest num		
2	6/3/26	P-2 FCFS		
3	6/3/26	P-3 - SJF		
4	6/3/26	P-4 - SRTF		
5	13/03/26	P-5 - priority		
6	13/03/26	P-6 - priority		
7	13/3/26	P-7 Round Robin		
8	20/3/26	Multilevel que - 8		
9	27/3/26	RMS scheduling - 9		
10	27/3/26	EDF scheduling - 10		
11	29/3/26	Proportional share scheduling		
12	24/4/26	Procedure Consumer		
13	24/4/26	Dining Philosopher		
15	8/5/26	Banker's algorithm		
Late submission				
incomplete (completed)				